

Crivo

Banco de dados

Diagrama de relacionamentos, tabelas, formato binário das células, versões e concorrência, consultas prontas e preparação do PostgreSQL.

Copyright (©) 2026 Erik Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 4
- PostgreSQL é a fonte da verdade 4
- Áreas do modelo 4
- Modo local (sem servidor) 4

Diagrama de relacionamentos 5
- Projetos e versões 5
- Paletas 5

Regras e convenções 6
- Convenções do modelo 6
- Integridade 6
- Exclusões em cascata 6
- Validação antes de gravar 7

Tabelas 8
- projeto 8
- camada 8
- quadro 9
- celula 9
- revisao 9
- paleta 10
- cor_paleta 10
- Tabelas do Django 10

Formato binário das células 12

Versões e concorrência 13
- Um salvamento passo a passo 13
- Restaurar uma versão 13
- Tamanho e retenção 13

Consultas prontas 14
- Projetos mais recentes 14
- Espaço ocupado por projeto 14
- Taxa de compressão das células 14

Cores de uma paleta 14
Versões de um projeto 15

Como preparar o banco **16**
Com Docker (recomendado) 16
PostgreSQL próprio 16
Backup e restauração 16
Desempenho 16

Evolução do esquema **17**

Capítulo 1

Visão geral

PostgreSQL é a fonte da verdade

Quando o Crivo roda com o servidor Django, tudo o que o editor salva vai para o PostgreSQL 17: projetos, camadas, quadros, o conteúdo de cada célula, as versões anteriores e as paletas. O editor no navegador nunca acessa o banco direto; ele fala só com a API, que valida cada projeto antes de gravar.

Sem variáveis de banco, o Django usa SQLite num arquivo local. O esquema é o mesmo, criado pelas mesmas migrações; só mudam os tipos físicos (por exemplo, `uuid` vira `char(32)` e `bytea` vira `BLOB`). SQLite é ótimo para uso pessoal; para várias pessoas ao mesmo tempo, use PostgreSQL.

Áreas do modelo

Área	Tabelas	Guarda
Projetos	`projeto`, `camada`, `quadro`, `celula`	O desenho: tamanho, ordem das camadas e quadros, e os pixels de cada combinação quadro × camada
Versões	`revisao`	Cópias comprimidas das versões anteriores de cada projeto
Paletas	`paleta`, `cor_paleta`	Paletas predefinidas e as criadas pelas pessoas
Django	`django_migrations`, `django_content_type`	Controle das migrações e tipos de conteúdo

Modo local (sem servidor)

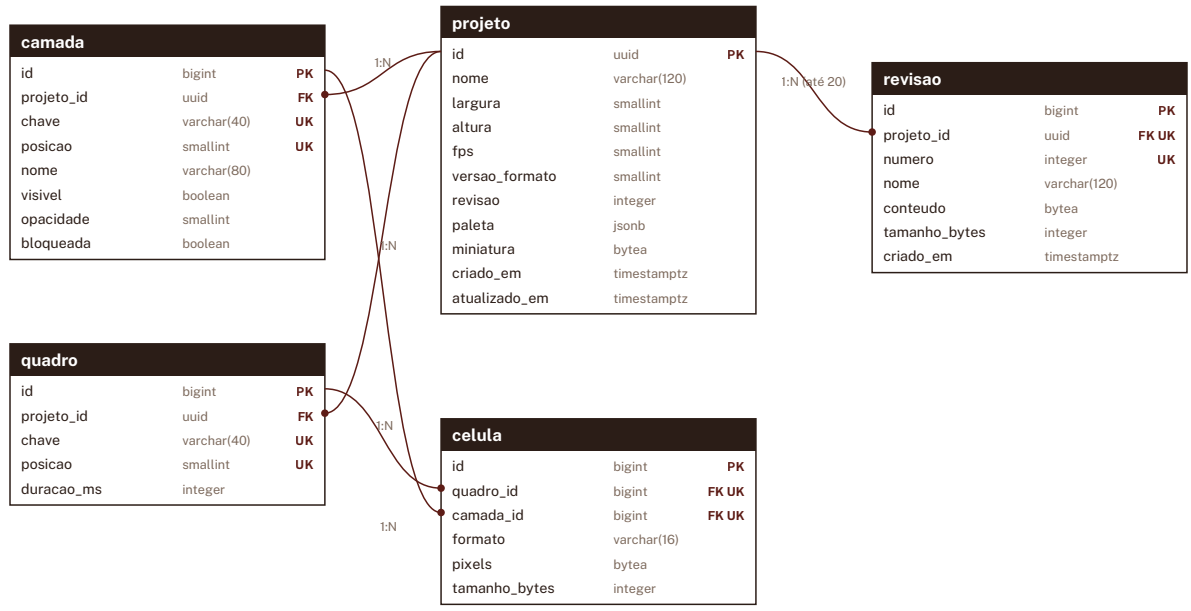
Sem API, o editor grava no próprio navegador, num esquema mais simples:

Onde	Chave	Conteúdo
IndexedDB `crivo`, loja `projetos`	`id` (texto `local-...`)	`{id, nome, dados, miniatura, atualizado_em}`, em que `dados` é o projeto no formato `.crivo`
IndexedDB `crivo`, loja `rascunho`	`atual`	O último estado do editor, para recuperar a sessão
`localStorage`	`crivo.paletas`	Lista das paletas criadas no navegador
`localStorage`	`crivo.*`	Preferências: ferramenta, pincel, tema, cores recentes, opções dos tons

Capítulo 2

Diagrama de relacionamentos

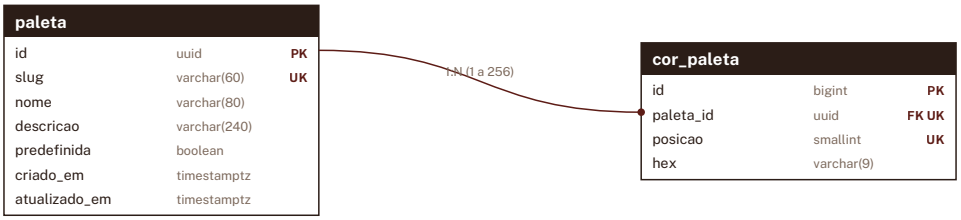
Projetos e versões



projeto 1:N camada · projeto 1:N quadro · quadro 1:N celula · camada 1:N celula · projeto 1:N revisao

Uma célula é a interseção de um quadro com uma camada. Num projeto com 3 camadas e 4 quadros há até 12 células, mas só as que têm algum pixel pintado são gravadas: uma célula ausente significa "tudo transparente".

Paletas



paleta 1:N cor_paleta, com a posição de cada cor

As paletas não se ligam aos projetos. Cada projeto guarda a sua própria lista de cores em `projeto.paleta` (JSON), para que apagar ou mudar uma paleta nunca altere um desenho salvo.

Capítulo 3

Regras e convenções

Convenções do modelo

- Nomes de tabelas e colunas em português, no singular e em minúsculas (`projeto`, `cor_paleta`).
- `projeto` e `paleta` usam `uuid` como chave primária: os endereços da API não revelam quantos projetos existem e podem ser gerados sem consultar o banco.
- As demais tabelas usam `bigint` gerado por identidade (`GENERATED BY DEFAULT AS IDENTITY`).
- `chave` em `camada` e `quadro` é o identificador criado pelo editor (por exemplo, `cmvihr...`). É única dentro do projeto e é o que liga as células no arquivo `.crivo`.
- `posicao` começa em 0. Nas camadas, 0 é a camada de baixo; nos quadros, 0 é o primeiro.
- Datas são `timestamptz`, em UTC no banco. O Django converte para `America/Sao_Paulo` (ou o fuso de `CRIVO_TIMEZONE`).

Integridade

Restrição	Tabela	Regra
`projeto_largura_valida`, `projeto_altura_valida`	projeto	Largura e altura entre 1 e 1024
`projeto_fps_valido`	projeto	FPS entre 1 e 60
`camada_chave_unica`, `camada_posicao_unica`	camada	Chave e posição únicas no projeto
`camada_opacidade_valida`	camada	Opacidade entre 0 e 100
`quadro_chave_unica`, `quadro_posicao_unica`	quadro	Chave e posição únicas no projeto
`celula_unica`	celula	No máximo uma célula por quadro e camada
`revisao_numero_unico`	revisao	Número de versão único no projeto
`cor_posicao_unica`	cor_paleta	Uma cor por posição da paleta
`cor_hex_valido`	cor_paleta	Texto `#rrggbb` ou `#rrggbbaa` em minúsculas
`slug` único	paleta	Identificador estável das paletas predefinidas (as criadas pelas pessoas têm `slug` nulo)

Exclusões em cascata

As chaves estrangeiras são criadas pelo Django como `DEFERRABLE INITIALLY DEFERRED`, e a exclusão em cascata é feita pelo próprio Django dentro da transação: excluir um projeto apaga as camadas, os quadros, as células e as versões; excluir uma paleta apaga as cores. Se você apagar linhas direto pelo SQL, apague primeiro as filhas (`celula`, depois `camada` e `quadro`, `revisao`) ou use `DELETE ... USING` na ordem certa.

Validação antes de gravar

A API não confia no navegador. Antes de qualquer gravação, `editor/servico.py` confere:

- o campo `formato` igual a `crivo`, tamanho entre 1 e 1024, FPS entre 1 e 60, de 1 a 64 camadas e de 1 a 512 quadros;
- identificadores de camadas e quadros presentes, com até 40 caracteres e sem repetição entre si;
- toda célula apontando para um quadro e uma camada que existem, sem duplicatas;
- o RLE de cada célula decodificando para exatamente largura × altura pixels;
- cores da paleta em hexadecimal válido e miniatura em PNG de até 512 KB.

Capítulo 4

Tabelas

Tipos conforme o PostgreSQL. Na coluna Nulo, "sim" indica que o campo aceita `NULL`.

projeto

Um desenho com suas dimensões e metadados. Os pixels ficam em `celula`.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária, gerada pela aplicação
`nome`	`varchar(120)`	não	Nome mostrado no editor e na lista
`largura`	`smallint`	não	Colunas da matriz (1 a 1024)
`altura`	`smallint`	não	Linhas da matriz (1 a 1024)
`fps`	`smallint`	não	Velocidade da animação (1 a 60)
`versao_formato`	`smallint`	não	Versão do formato `.crivo` usada (hoje 1)
`revisao`	`integer`	não	Contador de salvamentos, usado no controle de concorrência. Começa em 1
`paleta`	`jsonb`	não	Lista de cores do projeto em hexadecimal
`miniatura`	`bytea`	sim	PNG de até 128 px gerado pelo editor
`criado_em`	`timestamptz`	não	Criação
`atualizado_em`	`timestamptz`	não	Último salvamento (índice decrescente `projeto_atualizado_idx`)

camada

As camadas do projeto, na ordem de empilhamento. Valem para todos os quadros.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`projeto_id`	`uuid`	não	Projeto (FK, indexada)
`chave`	`varchar(40)`	não	Identificador da camada no editor; único no projeto
`posicao`	`smallint`	não	Ordem, 0 = camada de baixo; única no projeto
`nome`	`varchar(80)`	não	Nome da camada
`visivel`	`boolean`	não	Entra ou não na composição e nas exportações

Campo	Tipo	Nulo	Descrição
`opacidade`	`smallint`	não	0 a 100
`bloqueada`	`boolean`	não	O editor impede desenhar nela

quadro

Os quadros da animação, em ordem.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`projeto_id`	`uuid`	não	Projeto (FK, indexada)
`chave`	`varchar(40)`	não	Identificador do quadro no editor; único no projeto
`posicao`	`smallint`	não	Ordem, 0 = primeiro quadro; única no projeto
`duracao_ms`	`integer`	sim	Duração própria do quadro. Nulo = 1000 / fps

celula

Os pixels de um quadro numa camada, em RLE.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`quadro_id`	`bigint`	não	Quadro (FK, indexada)
`camada_id`	`bigint`	não	Camada (FK, indexada)
`formato`	`varchar(16)`	não	Codificação de `pixels`. Hoje sempre `rle-rgba`
`pixels`	`bytea`	não	Pares (corrida em varint, cor RGBA), ver o capítulo seguinte
`tamanho_bytes`	`integer`	não	Tamanho de `pixels`, para relatórios sem ler o conteúdo

revisao

Versões anteriores de um projeto. Cada salvamento guarda aqui o estado que estava no banco antes dele.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`projeto_id`	`uuid`	não	Projeto (FK, indexada)
`numero`	`integer`	não	Valor de `projeto.revisao` naquele momento; único no projeto

Campo	Tipo	Nulo	Descrição
`nome`	`varchar(120)`	não	Nome do projeto naquela versão
`conteudo`	`bytea`	não	O projeto inteiro no formato `.crivo`, em JSON comprimido com zlib
`tamanho_bytes`	`integer`	não	Tamanho de `conteudo`
`criado_em`	`timestamptz`	não	Quando a versão foi guardada

paleta

Paletas predefinidas (criadas pelas migrações) e paletas das pessoas.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`slug`	`varchar(60)`	sim	Identificador estável das predefinidas (`game-boy`, `nes`…); único
`nome`	`varchar(80)`	não	Nome mostrado
`descricao`	`varchar(240)`	não	Texto curto, vazio nas paletas das pessoas
`predefinida`	`boolean`	não	A API recusa alterar ou excluir quando verdadeiro
`criado_em`	`timestamptz`	não	Criação
`atualizado_em`	`timestamptz`	não	Última alteração

cor_paleta

As cores de cada paleta, na ordem.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`paleta_id`	`uuid`	não	Paleta (FK, indexada)
`posicao`	`smallint`	não	Ordem da cor, a partir de 0; única na paleta
`hex`	`varchar(9)`	não	`#rrggbb` ou `#rrggbbaa`, minúsculo

Tabelas do Django

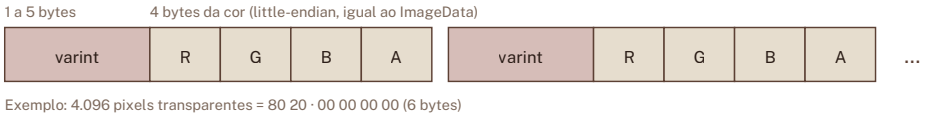
Tabela	Para que serve
`django_migrations`	Registra quais migrações já rodaram (`editor.0001_inicial`, `editor.0002_paletas_predefinidas` e as do Django)

Tabela	Para que serve
`django_content_type`	Catálogo interno de modelos do Django

Capítulo 5

Formato binário das células

A coluna `celula.pixels` usa o mesmo formato que o editor (JavaScript), o servidor (Python) e a ferramenta em Java leem e escrevem. A matriz é percorrida linha por linha, da esquerda para a direita, e cada sequência de pixels iguais vira um par:



Exemplo: 4.096 pixels transparentes = 80 20 · 00 00 00 00 (6 bytes)

Parte	Tamanho	Conteúdo
Corrida	1 a 5 bytes	Quantos pixels seguidos têm a cor. Varint: 7 bits por byte, do menos para o mais significativo; o bit 8 ligado indica que há outro byte
Cor	4 bytes	R, G, B e A, nessa ordem. A = 0 é transparente

A soma das corridas é sempre largura × altura; a API recusa células que não fecham. No arquivo `.crivo` e na API, os mesmos bytes viajam em base64 no campo `rle`. Para ler os pixels direto no banco, há duas opções:

- pela API, em `GET /api/v1/projetos/{id}/exportar`, que já decodifica, compõe as camadas visíveis e devolve a matriz em JSON, CSV, JavaScript ou Python;
- pelo código, com `editor.rle.decodificar(bytes, largura * altura)`, que devolve um `array('I')` com um inteiro por pixel (`R | G << 8 | B << 16 | A << 24`).

Capítulo 6

Versões e concorrência

Um salvamento passo a passo

1. O editor envia `PUT /api/v1/projetos/{id}` com o projeto e o número de `revisao` que ele conhece.
2. Numa transação, a API trava a linha do projeto com `SELECT ... FOR UPDATE`. Outro salvamento do mesmo projeto espera esse terminar.
3. Se a revisão enviada for diferente de `projeto.revisao`, a transação é desfeita e a resposta é 409, com a revisão atual. Nada é gravado.
4. O estado atual do banco vira uma linha em `revisao` (JSON do `.crivo` comprimido com zlib). Se houver mais de `CRIVO_MAX_REVISOES` versões, as mais antigas são apagadas.
5. As camadas e os quadros antigos são removidos (com as células) e os novos são inseridos com `bulk_create`, em lotes.
6. `projeto.revisao` aumenta 1, e a resposta devolve o novo número, que o editor guarda para o próximo salvamento.

> Como a troca acontece dentro de uma única transação, quem lê o projeto ao mesmo tempo vê a versão antiga inteira ou a nova inteira, nunca uma mistura.

Restaurar uma versão

`POST /api/v1/projetos/{id}/revisoes/{n}/restaurar` descomprime a versão `n` e faz um salvamento normal com ela. Por isso a versão que estava em uso também vai para o histórico, e restaurar pode ser desfeito restaurando de novo.

Tamanho e retenção

Cada versão guarda o projeto inteiro, mas comprimido duas vezes (RLE dentro do JSON e zlib por fora). Um sprite 32×32 com 4 quadros costuma ocupar menos de 2 KB por versão. Para guardar mais ou menos versões, ajuste `CRIVO_MAX_REVISOES`; o corte acontece no próximo salvamento de cada projeto.

Capítulo 7

Consultas prontas

Projetos mais recentes

```
SELECT p.nome, p.largura || 'x' || p.altura AS tamanho,
       (SELECT count(*) FROM quadro q WHERE q.projeto_id = p.id) AS quadros,
       (SELECT count(*) FROM camada c WHERE c.projeto_id = p.id) AS camadas,
       p.revisao, p.atualizado_em
FROM projeto p
ORDER BY p.atualizado_em DESC
LIMIT 20;
```

Espaço ocupado por projeto

```
SELECT p.nome,
       coalesce(sum(ce.tamanho_bytes), 0) AS bytes_pixels,
       (SELECT coalesce(sum(r.tamanho_bytes), 0) FROM revisao r WHERE r.projeto_id = p.id) AS
bytes_versoes,
       coalesce(length(p.miniatura), 0) AS bytes_miniatura
FROM projeto p
LEFT JOIN quadro q ON q.projeto_id = p.id
LEFT JOIN celula ce ON ce.quadro_id = q.id
GROUP BY p.id
ORDER BY bytes_pixels DESC;
```

Taxa de compressão das células

```
SELECT p.nome, q.posicao + 1 AS quadro, c.nome AS camada,
       p.largura * p.altura * 4 AS bytes_brutos,
       ce.tamanho_bytes,
       round(100.0 * ce.tamanho_bytes / (p.largura * p.altura * 4), 1) AS percentual
FROM celula ce
JOIN quadro q ON q.id = ce.quadro_id
JOIN camada c ON c.id = ce.camada_id
JOIN projeto p ON p.id = q.projeto_id
ORDER BY percentual DESC;
```

Cores de uma paleta

```
SELECT pa.nome, cp.posicao, cp.hex
FROM paleta pa
JOIN cor_paleta cp ON cp.paleta_id = pa.id
WHERE pa.slug = 'pico-8'
ORDER BY cp.posicao;
```

Versões de um projeto

```
SELECT r.numero, r.nome, r.tamanho_bytes, r.criado_em
FROM revisao r
JOIN projeto p ON p.id = r.projeto_id
WHERE p.nome = 'Poção'
ORDER BY r.numero DESC;
```

Capítulo 8

Como preparar o banco

Com Docker (recomendado)

O `docker-compose.yml` já sobe o PostgreSQL 17 com o banco, o usuário e a senha do `.env`, num volume chamado `dados-postgres`. A aplicação espera o banco ficar saudável, roda `migrate` e começa a atender. Não há nada a fazer além de rodar `iniciar.cmd` ou `./iniciar.sh`.

PostgreSQL próprio

```
CREATE ROLE crivo WITH LOGIN PASSWORD 'troque-esta-senha';
CREATE DATABASE crivo OWNER crivo ENCODING 'UTF8';
```

```
export CRIVO_DB_ENGINE=postgresql
export CRIVO_DB_HOST=localhost CRIVO_DB_PORT=5432
export CRIVO_DB_NAME=crivo CRIVO_DB_USER=crivo CRIVO_DB_PASSWORD=troque-esta-senha
python manage.py migrate
```

Qualquer PostgreSQL a partir da versão 14 atende. Os gerenciados (Amazon RDS, Azure, Google Cloud SQL, Supabase, Neon) funcionam do mesmo jeito; se o serviço exigir SSL, acrescente `sslmode=require` nas opções de conexão em `crivo/settings.py`.

Backup e restauração

```
docker compose exec postgres pg_dump -U crivo -d crivo -Fc > crivo.dump

docker compose exec -T postgres pg_restore -U crivo -d crivo --clean --if-exists < crivo.dump
```

O primeiro comando gera um backup compacto; o segundo restaura por cima do banco atual. Como os projetos também podem ser baixados um a um como `.crivo`, outra forma simples de backup é guardar esses arquivos.

Desempenho

- As listas de projetos usam o índice `projeto_atualizado_idx` e não leem a coluna `miniatura`.
- Abrir um projeto lê as células com uma única consulta por projeto.
- Células e versões são `bytea`. Valores grandes vão automaticamente para o armazenamento TOAST do PostgreSQL, comprimidos, sem pesar nas varreduras das outras colunas.
- Gravações usam `bulk_create` em lotes de 200 células, dentro de uma transação.

Capítulo 9

Evolução do esquema

O esquema é criado e alterado só por migrações do Django, na pasta `editor/migrations`:

Migração	O que faz
`0001_inicial`	Cria as sete tabelas, as chaves, os índices e as restrições
`0002_paletas_predefinidas`	Cria as paletas Crivo, Game Boy, NES, Pico-8, CGA, CGA modo 4, Commodore 64 e 1 bit, a partir de `editor/paletas.py`

Para mudar o modelo, altere `editor/models.py`, gere a migração e aplique:

```
python manage.py makemigrations editor
python manage.py migrate
python manage.py makemigrations --check --dry-run
```

O último comando é o mesmo que a integração contínua roda: ele falha se o código e as migrações estiverem fora de sincronia. Se a mudança alterar o formato do arquivo, aumente `VERSAO_FORMATO` em `js/nucleo/formato.js` e trate a versão antiga em `desserializar`, para que projetos já salvos continuem abrindo.